

Student 14 **Jeyakannan L** [192525376RD](#)

5 / 5 submitted

Q1. Smart Document Editor using String, StringBuilder and StringBuffer

CODE

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        String paragraph = sc.nextLine();
        int characters = paragraph.length();
        String[] words = paragraph.trim().split("\\s+");
        int wordCount = words.length;
        int sentenceCount = 0;
        for (int i = 0; i < paragraph.length(); i++) {
            char ch = paragraph.charAt(i);
            if (ch == '.' || ch == '!' || ch == '?') {
                sentenceCount++;
            }
        }
        String oldWord = sc.nextLine();
        String newWord = sc.nextLine();
        String replacedParagraph = paragraph.replace(oldWord, newWord);
        StringBuffer history = new StringBuffer();
        history.append("Original: ").append(paragraph).append("\n");
        history.append("Replaced \"".append(oldWord).append("\" with \""
            .append(newWord).append("\"");
        String[] sentences = replacedParagraph.split("(?<=[.!?])");

        StringBuilder reversedDocument = new StringBuilder();

        for (String s : sentences) {
            s = s.trim();
            if (s.length() == 0)
                continue;

            char end = s.charAt(s.length() - 1);

            if (end == '.' || end == '!' || end == '?') {
                s = s.substring(0, s.length() - 1);
            } else {
                end = ' ';
            }

            StringBuilder temp = new StringBuilder(s);
            temp.reverse();

            reversedDocument.append(temp);

            if (end != ' ')
                reversedDocument.append(end);

            reversedDocument.append(" ");
        }

        StringBuilder finalDoc = new StringBuilder();
        finalDoc.append(reversedDocument.toString().trim());

        System.out.println("Original Document:");
        System.out.println(paragraph);
        System.out.println("*****");
        System.out.println("Characters: " + characters);
        System.out.println("Words: " + wordCount);
        System.out.println("Sentences: " + sentenceCount);
    }
}
```

Report

```
        System.out.println("*****");
        System.out.println("Editing History:");
        System.out.println(history);
        System.out.println("*****");
        System.out.println("Final Document:");
        System.out.println(finalDoc);
        System.out.println("*****");
    }
}
```

INPUT

```
Java is easy. I like Java programming.
Java
Python
```

OUTPUT

(no output)

Q2. Hospital Token Management System using List Interface

CODE

```
import java.util.*;

public class Patient {
    int id;
    String name;
    int age;
    String department;
    Patient(int id, String name, int age, String department) {
        this.id = id;
        this.name = name;
        this.age = age;
        this.department = department;
    }
    public String toString() {
        return "ID: " + id + ", Name: " + name + ", Age: " + age + ", Department: " + department;
    }
    static List<Patient> patients = new ArrayList<Patient>();
    static void registerPatient(int id, String name, int age, String department) {
        patients.add(new Patient(id, name, age, department));
    }
    static void displayPatients() {
        System.out.println("Patients in Registration Order:");
        for (Patient p : patients) {
            System.out.println(p);
        }
    }
    static void updatePatient(int id, String name, int age, String department) {
        for (Patient p : patients) {
            if (p.id == id) {
                p.name = name;
                p.age = age;
                p.department = department;
                System.out.println("Patient Updated Successfully.");
                return;
            }
        }
        System.out.println("Patient ID Not Found.");
    }
    static void removePatient(int id) {
        Iterator<Patient> it = patients.iterator();
        while (it.hasNext()) {
            Patient p = it.next();
            if (p.id == id) {
                it.remove();
                System.out.println("Cancelled Appointment Removed.");
                return;
            }
        }
        System.out.println("Patient ID Not Found.");
    }
    static void displayDepartment(String department) {
        System.out.println("Patients in " + department + " Department:");
        for (Patient p : patients) {
            if (p.department.equalsIgnoreCase(department)) {
                System.out.println(p);
            }
        }
    }
    static void firstAndLastPatient() {
```

Report

```
        if (patients.size() == 0) {
            System.out.println("No Patients Registered.");
            return;
        }
        System.out.println("First Registered Patient:");
        System.out.println(patients.get(0));
        System.out.println("Last Registered Patient:");
        System.out.println(patients.get(patients.size() - 1));
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        for (int i = 0; i < n; i++) {
            int id = sc.nextInt();
            String name = sc.next();
            int age = sc.nextInt();
            String department = sc.next();

            registerPatient(id, name, age, department);
        }
        displayPatients();
        int updateId = sc.nextInt();
        String newName = sc.next();
        int newAge = sc.nextInt();
        String newDepartment = sc.next();
        updatePatient(updateId, newName, newAge, newDepartment);
        int removeId = sc.nextInt();
        removePatient(removeId);
        String searchDepartment = sc.next();
        displayDepartment(searchDepartment);
        firstAndLastPatient();
        sc.close();
    }
}
```

INPUT

```
5
301 Aadhira 28 Cardiology
302 Kavin 45 Neurology
303 Meera 34 Orthopedics
304 Rithvik 52 Oncology
305 Diya 23 Pediatrics
303 MeeraPriya 35 Dermatology
302
Dermatology
```

OUTPUT

(no output)

Q3. Airport Boarding Queue Management using Queue Interface

CODE

```
import java.util.*;

public class AirportQueue {

    static Queue<String> queue = new LinkedList<String>();
    static Queue<String> vipQueue = new PriorityQueue<String>();

    static void addPassenger(String name) {
        queue.offer(name);
    }

    static void addVIP(String name) {
        vipQueue.offer(name);
    }

    static void displayQueue() {
        System.out.println("Current Boarding Queue");
        int i = 1;
        for (String s : queue) {
            System.out.println(i + ". " + s);
            i++;
        }
    }

    static void nextPassenger() {
        if (queue.isEmpty()) {
            System.out.println("Queue is Empty");
        } else {
            System.out.println("Next Passenger: " + queue.peek());
        }
    }

    static void boardPassenger() {
        if (queue.isEmpty()) {
            System.out.println("Queue is Empty");
        } else {
            System.out.println("Boarded Passenger: " + queue.poll());
        }
    }

    static void remainingPassengers() {
        System.out.println("Remaining Passengers");
        int i = 1;
        for (String s : queue) {
            System.out.println(i + ". " + s);
            i++;
        }
    }

    static void displayVIP() {
        System.out.println("VIP Passengers");
        int i = 1;
        for (String s : vipQueue) {
            System.out.println(i + ". " + s);
            i++;
        }
    }

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
```

```
        int n = sc.nextInt();

        for (int i = 0; i < n; i++) {
            String name = sc.next();
            addPassenger(name);
        }

        displayQueue();

        nextPassenger();

        boardPassenger();

        remainingPassengers();

        int v = sc.nextInt();

        for (int i = 0; i < v; i++) {
            String name = sc.next();
            addVIP(name);
        }

        displayVIP();

        sc.close();
    }
}
```

INPUT

```
5
Arjun
Nisha
Rahul
Divya
Karthik
3
Zara
Aakash
Vikram
```

OUTPUT

(no output)

Q4. Digital Certificate Verification using Set Interface

CODE

```
import java.util.*;

public class CertificateVerification {
    LinkedHashSet<String> certificates = new LinkedHashSet<String>();

    void addCertificate(String id) {
        if (certificates.add(id))
            System.out.println(id + " Added");
        else
            System.out.println(id + " Duplicate Certificate");
    }

    void verifyCertificate(String id) {
        if (certificates.contains(id))
            System.out.println("Certificate Found");
        else
            System.out.println("Certificate Not Found");
    }

    void displayInsertionOrder() {
        System.out.println("Certificates in Insertion Order:");
        for (String id : certificates) {
            System.out.println(id);
        }
    }

    void displaySortedOrder() {
        TreeSet<String> sorted = new TreeSet<String>(certificates);
        System.out.println("Certificates in Sorted Order:");
        for (String id : sorted) {
            System.out.println(id);
        }
    }

    void totalCertificates() {
        System.out.println("Total Unique Certificates: " + certificates.size());
    }

    void removeCertificate(String id) {
        if (certificates.remove(id))
            System.out.println("Certificate Removed");
        else
            System.out.println("Certificate Not Found");
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        CertificateVerification obj = new CertificateVerification();
        int n = sc.nextInt();
        for (int i = 0; i < n; i++) {
            String id = sc.next();
            obj.addCertificate(id);
        }
        obj.displayInsertionOrder();
        String verifyId = sc.next();
        obj.verifyCertificate(verifyId);
        obj.displaySortedOrder();
        obj.totalCertificates();
        String removeId = sc.next();
        obj.removeCertificate(removeId);
        System.out.println("Final Certificate List:");
        obj.displayInsertionOrder();
        sc.close();
    }
}
```

INPUT

```
7
CERT101
CERT205
CERT110
```



```
CERT205
CERT350
CERT450
CERT101
CERT350
CERT110
```

OUTPUT

```
(no output)
```

Q5. Smart Inventory Management using Map Interface

CODE

```
import java.util.*;

public class Inventory {
    HashMap<Integer, Product> products = new HashMap<Integer, Product>();
    class Product {
        int id;
        String name;
        String category;
        int quantity;
        double price;
        Product(int id, String name, String category, int quantity, double price) {
            this.id = id;
            this.name = name;
            this.category = category;
            this.quantity = quantity;
            this.price = price;
        }
    }
    void addProduct(int id, String name, String category, int quantity, double price) {
        products.put(id, new Product(id, name, category, quantity, price));
    }
    void updateQuantity(int id, int quantity) {
        if (products.containsKey(id)) {
            products.get(id).quantity = quantity;
            System.out.println("Quantity Updated");
        } else {
            System.out.println("Product Not Found");
        }
    }
    void searchProduct(int id) {
        if (products.containsKey(id)) {
            Product p = products.get(id);
            System.out.println("Product ID : " + p.id);
            System.out.println("Name      : " + p.name);
            System.out.println("Category   : " + p.category);
            System.out.println("Quantity  : " + p.quantity);
            System.out.println("Price     : " + p.price);
        } else {
            System.out.println("Product Not Found");
        }
    }
    void totalValue() {
        double total = 0;
        for (Product p : products.values()) {
            total = total + (p.quantity * p.price);
        }
        System.out.println("Total Inventory Value = " + total);
    }
    void displayCategory(String category) {
        System.out.println("Products in " + category + " Category");
        for (Product p : products.values()) {
            if (p.category.equalsIgnoreCase(category)) {
                System.out.println(p.id + " " + p.name + " " + p.quantity + " " + p.price);
            }
        }
    }
    void removeProduct(int id) {
        if (products.remove(id) != null)
            System.out.println("Product Removed");
        else
            System.out.println("Product Not Found");
    }
}
```

Report

```
void displayProducts() {
    System.out.println("Available Products");
    for (Product p : products.values()) {
        System.out.println(p.id + " " + p.name + " " + p.category + " " + p.quantity + " " + p.price);
    }
}

public static void main(String args[]) {
    Scanner sc = new Scanner(System.in);
    Inventory obj = new Inventory();
    int n = sc.nextInt();
    for (int i = 0; i < n; i++) {
        int id = sc.nextInt();
        String name = sc.next();
        String category = sc.next();
        int quantity = sc.nextInt();
        double price = sc.nextDouble();
        obj.addProduct(id, name, category, quantity, price);
    }
    int updateId = sc.nextInt();
    int newQty = sc.nextInt();
    obj.updateQuantity(updateId, newQty);
    int searchId = sc.nextInt();
    obj.searchProduct(searchId);
    obj.totalValue();
    String category = sc.next();
    obj.displayCategory(category);
    int removeId = sc.nextInt();
    obj.removeProduct(removeId);
    obj.displayProducts();
    sc.close();
}
}
```

INPUT

```
5
301 Football Sports 15 1200
302 CricketBat Sports 10 2500
303 TennisBall Sports 40 150
304 BadmintonRacket Sports 12 1800
305 Volleyball Sports 20 900
302
18
304
Sports
305
```

OUTPUT

(no output)